Lab No. 05

Managing Processes on Linux

Objective:

This lab is designed to demonstarate the basic task management related activities.

Activity Outcomes:

On completion of this lab students will be able to:

- How to manage processes in Linux

Instructor Notes

As pre-lab activity, read Chapter 10 from the book “The Linux Command Line”, William E. Shotts, Jr.

1) Useful Concepts

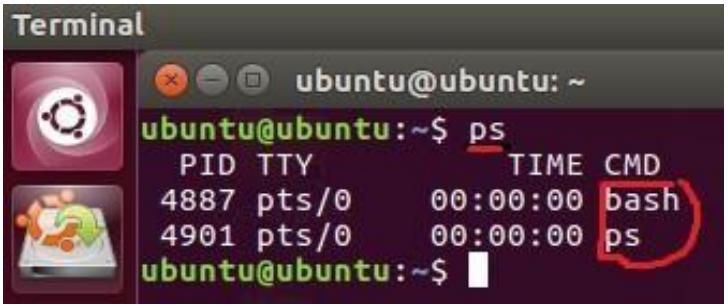
Process management in Linux

A process is the instance of a computer program that is being executed. While a computer program is a passive collection of instructions, a process is the actual execution of those instructions. Several processes may be associated with the same program; for example, opening up several instances of the same program often results in more than one process being executed. When a system starts up, the kernel initiates a few of its own activities as processes and launches a program called init. init, in turn, runs a series of shell scripts (located in /etc) called init scripts, which start all the system services. Many of these services are implemented as daemon programs, programs that just sit in the background and do their thing without having any user interface. So even if we are not logged in, the system is at least a little busy performing routine stuff.

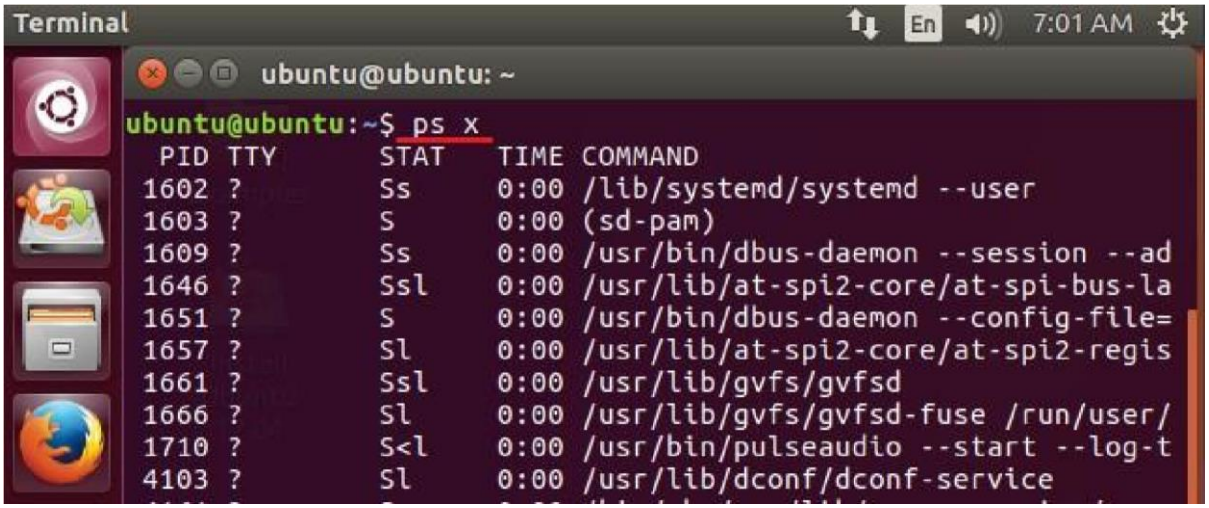
The kernel maintains information about each process to help keep things organized. For example, each process is assigned a number called a process ID or PID. PIDs are assigned in ascending order, with init always getting PID 1. The kernel also keeps track of the memory assigned to each process, as well as the processes' readiness to resume execution. Like files, processes also have owners and user IDs, effective user IDs, etc. In the following, we will discuss the most common commands; available in Linux to mange processes.

Displaying Processes in the System

Static view: The most commonly used command to view processes is ps. This command displays the processes for the current shell. We can use the ps command as given below



We can display all of the processes owned by the current user by using the x option.



Here, a new column is also added in the output that is STAT. This column shows the currentstate of the process. The most common states are given below.

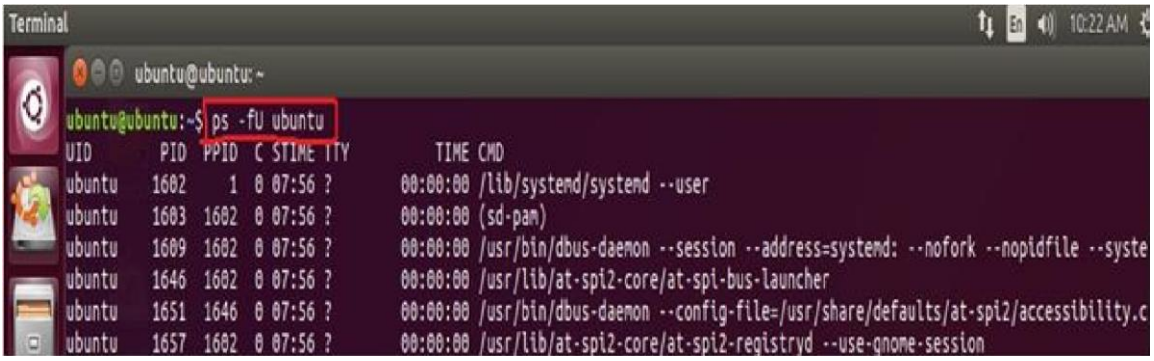
State	Meaning
R	Running
S	Sleeping or waiting for an event such as keystroke
D	Uninterruptible Sleep. Process is waiting for I/O such as a disk drive
T	Stopped
Z	A difunctional or zombie process
l	multithreaded
s	Session leader
+	Foreground process
<	A high priority process
N	A low priority process

Following are the most common option that can be used with ps command.

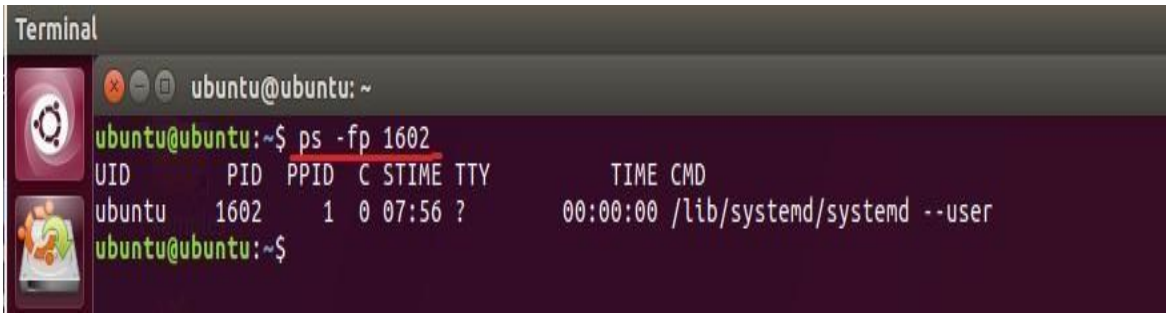
Option	Description
-A or -e	Display every active process on a Linux system
-x	Display all of the processes owned by the user
-F	Perform a full-format listing
-U	Select by real user ID or name
-u	Select by effective user ID or name
-p	Select process by pid
-r	Display only running processes

-L	Show number of threads in a process
-G	Show processes by Group

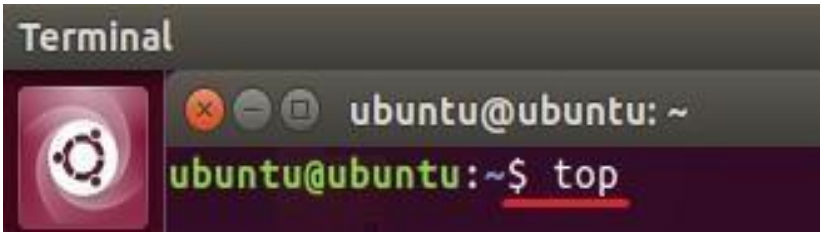
In the following example, we display processes that are related to the user ubuntu.



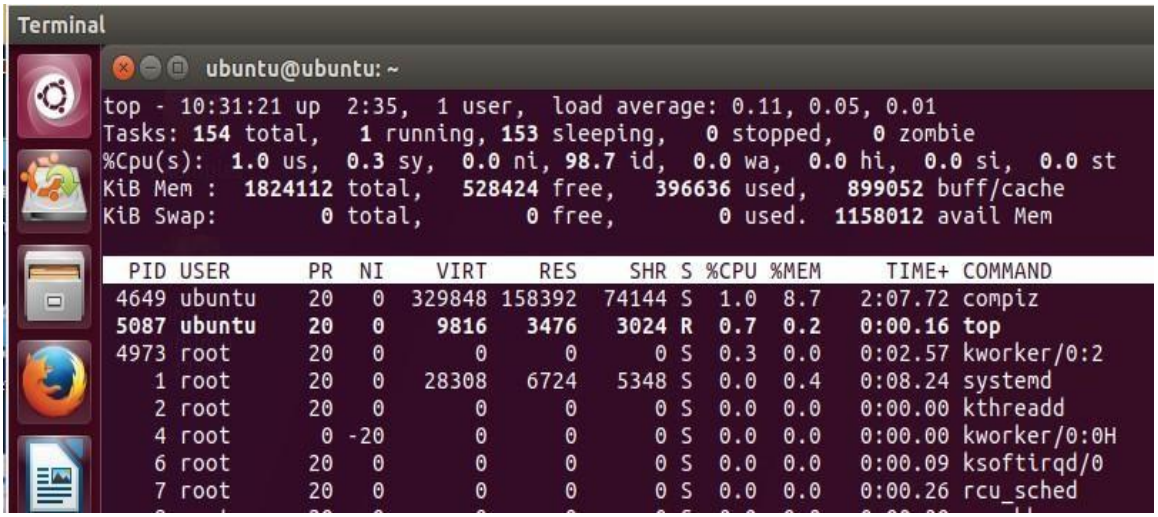
Now, we select a process with id 1602



Dynamic view: The top command is the traditional way to view your system's resource usage and see the processes that are taking up the most system resources. Top displays a list of processes, with the ones using the most CPU at the top. An improved version of top command is htop but it is usually not pre-installed in most distributions. When a top program is running, we can highlight the running programs by pressing **z**. we can quit the top program by press **q** or **Ctrl + c**.
In the following example we display the dynamic view of the system process and resource usage.



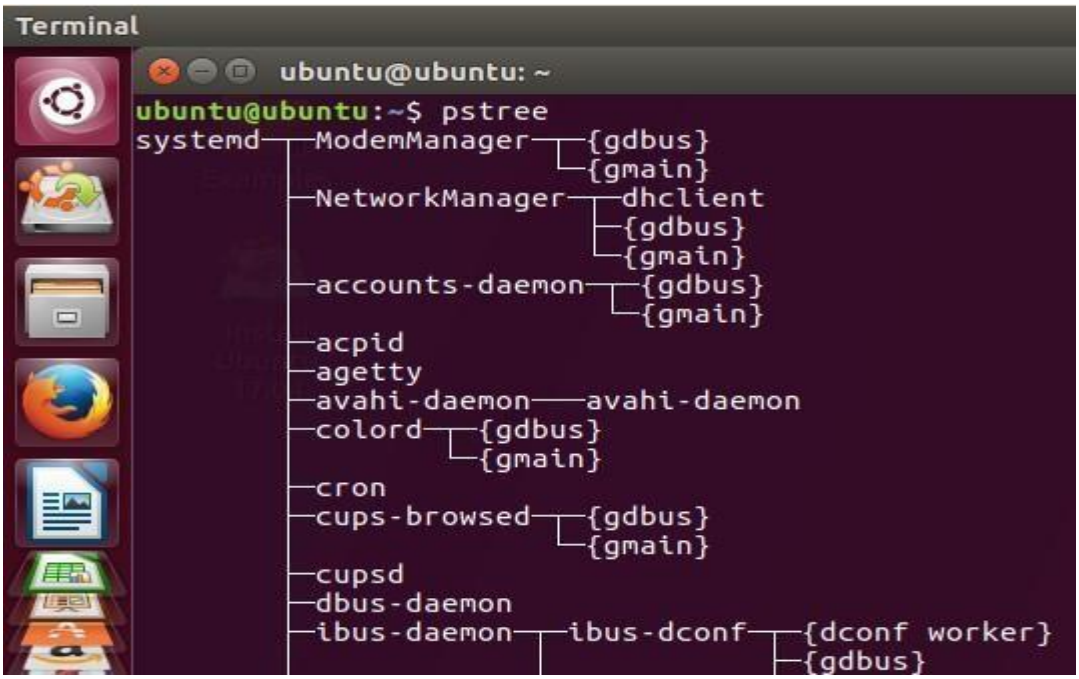
The output of the above command is given below:



Some of the basic options available with top commands are given below

Options	Description
-u	display specific User process details
-d	To set the screen refresh frequency

Displaying processes in Treelike structure: The pstree command is used to display processes in treelike structure showing the parent/child relationships between processes.



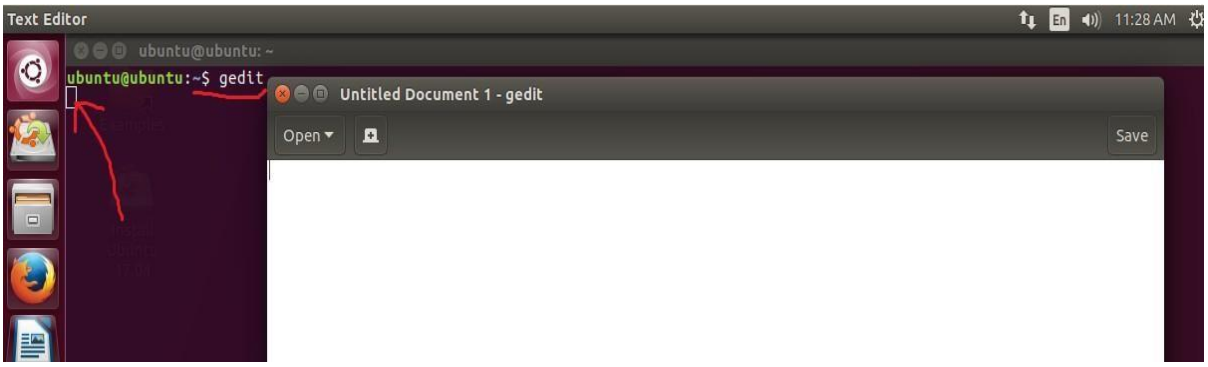
Interrupting A Process

A program can be interrupted by pressing Ctrl + C. This will interrupt the given processes and stop the process. Ctrl+C essentially sends a SIGINT signal from the controlling terminal to the process, causing it to be killed.

Putting a Process in the Background

A foreground process is any command or task you run directly and wait for it to complete. Unlike with a foreground process, the shell does not have to wait for a background process to end before it can run more processes. Normally, we start a program by entering its name in the CLI. However, if we want to start a program in background, we will put an & after its name.

In the following example, first we open the gedit program normally as given below.

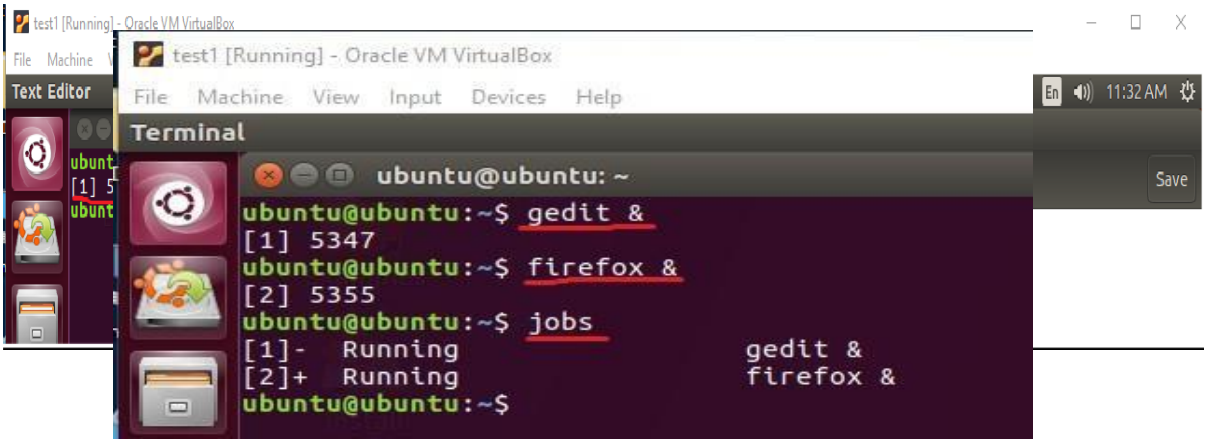


It can be noted that gedit is opened as foreground process and control does not returns to terminal unless it is closed. Now, we start the gedit again as a background process.

It can be seen that after starting the gedit program control returns to the terminal and user can interact with both terminal and gedit.

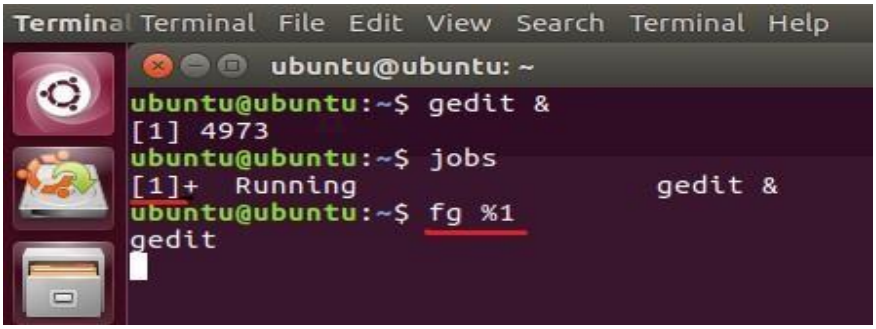
jobs command

The shell's job control facility also gives us a way to list the jobs that have been launched from our terminal. Using the **jobs** command, we can see this list.In the following example, we first launch two jobs in background and then use the jobs command to view the running jobs.



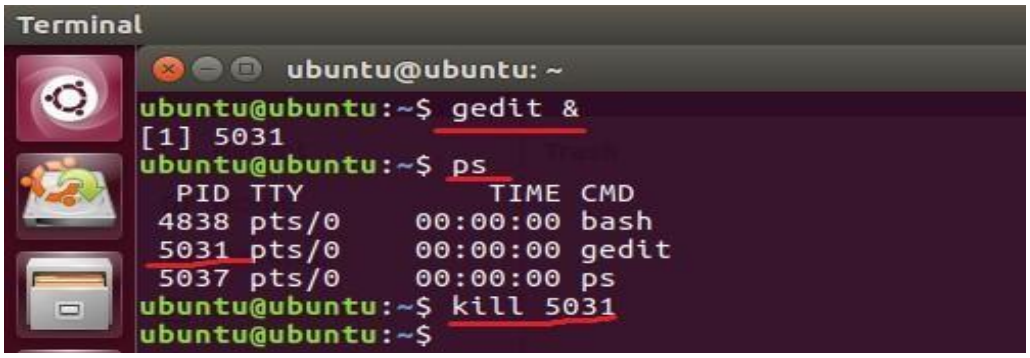
Bringing a process to the foreground

A process in the background is immune from keyboard input, including any attempt to interrupt it with a Ctrl-c. fg command is used to bring a process to the foreground. In the following example, we start the gedit editor in background. Then use the jobs command to see the list of jobs launched and then, bring this process to the foreground.



Killing a process

We can kill a process using the kill command. To kill a process, we provide the process id as an argument (We could have also specified the process using a jobspec). In the following example, we start the gedit program and then kill it using kill command.



The kill command doesn't exactly "kill" processes, rather it sends them *signals*. Signals are one of several ways that the operating system communicates with programs. Programs, in turn, "listen" for signals and may act upon them as they are received. Following are most common signals that can be send with kill command.

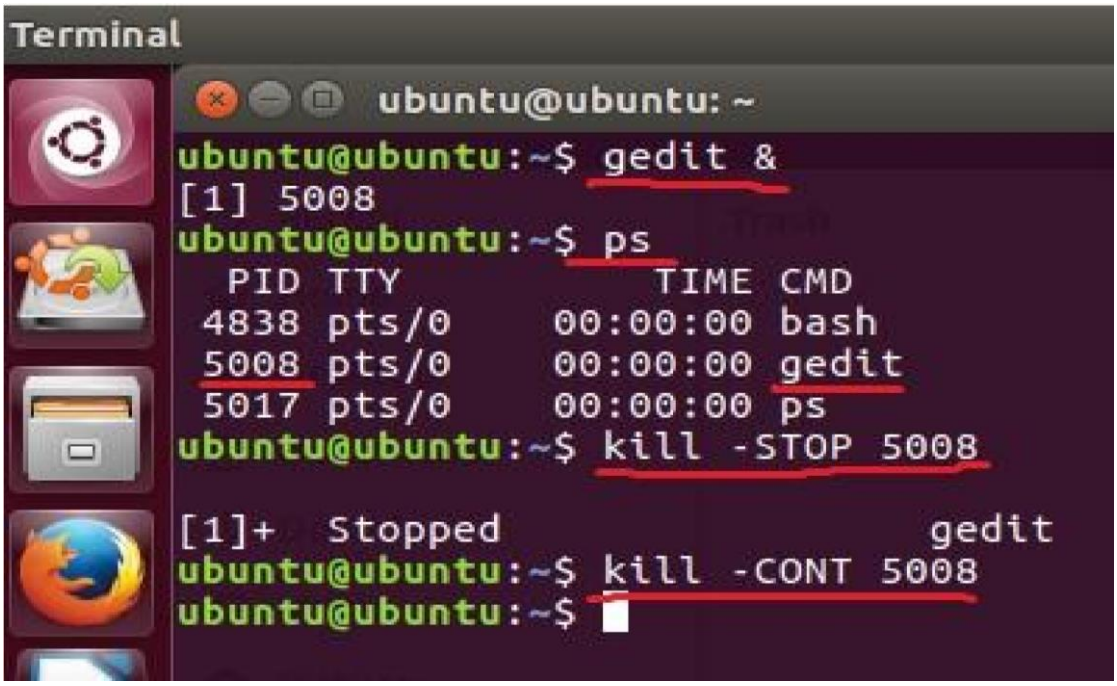
Sgnal	Meaning
INT	INT Interrupt. Performs the same function as the Ctrl-c key sent from the terminal. It will usually terminate a program.
TERM	Terminate. This is the default signal sent by the kill command. If a program is still "alive" enough to receive signals, it will terminate.
STOP	Stop. This signal causes a process to pause without terminating.
CONT	Continue. This will restore a process after a STOP signal.

Pausing a process

Linux allows you to pause a running process rather than quitting or killing it. Pausing a process just suspends all of its operation so it stops using any of your processor power even while it still resides in memory. This may be useful when you want to run some sort of a processor intensive task, but don't wish

to completely terminate another process you may have running. Pausing it would free up valuable processing time while you need it, and then continue it afterwards.

We can paus a process by using kill command with STOP option. The process id is required as an argument in kill command. In the following example, we start the gedit program in the background. Then we find the pid of gedit using ps command and then; we pause the process using kill command. Later, we can resume the process by using the kill command with CONT option.



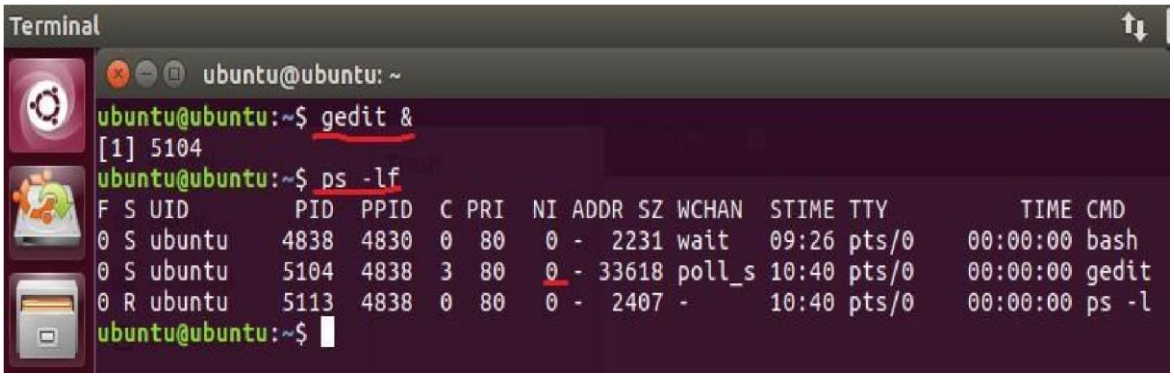
Changing process priority

Every running process in Linux has a priority assigned to it. We can change the process priority using nice and renice utility. Nice command will launch a process with a user defined scheduling priority. Renice command will modify the scheduling priority of a running process. In Linux system priorities are 0 to 139 in which 0 to 99 for real time and 100 to 139 for users. nice value range is -20 to +19 where -20 is highest, 0 default and +19 is lowest. Relation between nice value and priority is:

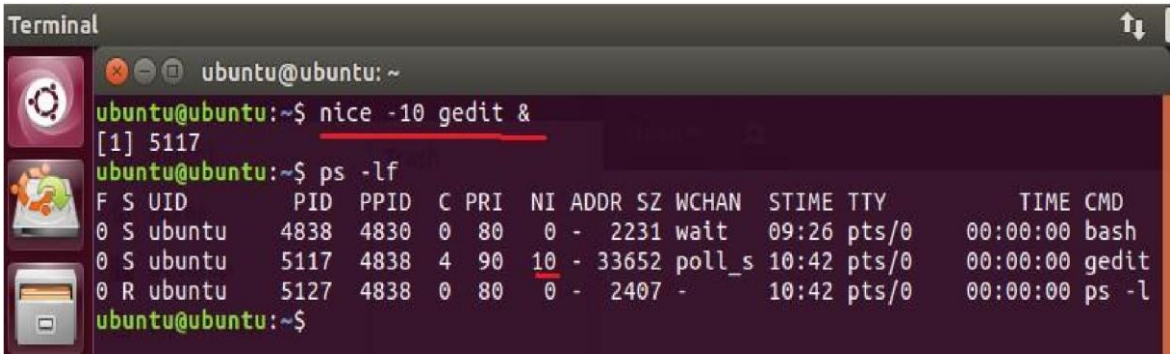
PR = 20 + NI

So, the value of PR = 20 + (-20 to +19) is 0 to 39 that maps to 100-139.

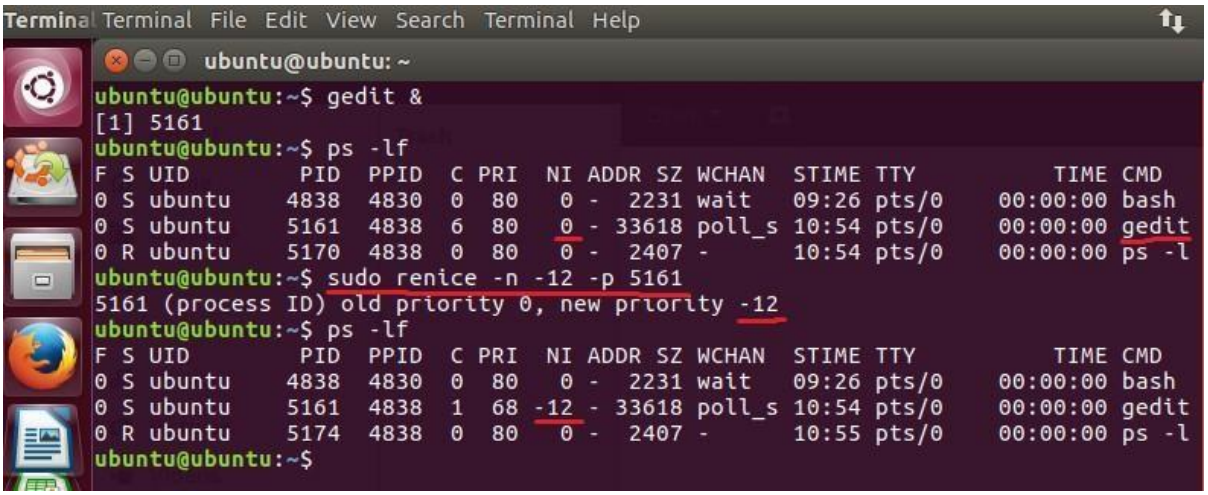
In the following example, we start the gedit program in background and see its nice value using ps command.



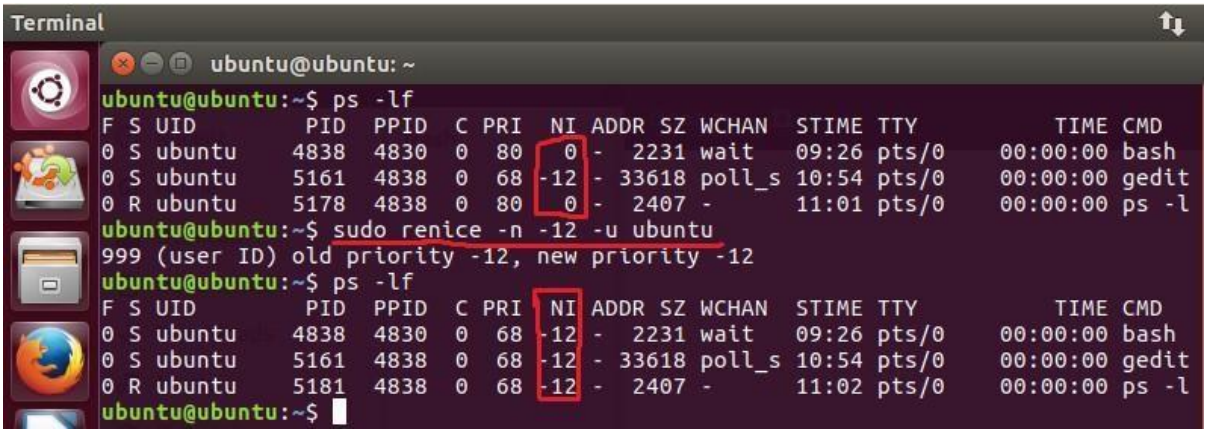
Now, we start the gedit program again using nice command.



We can change the priority of a running process using renice command. In the following example, we first start the gedit program in background. Then we change its priority using renice command.

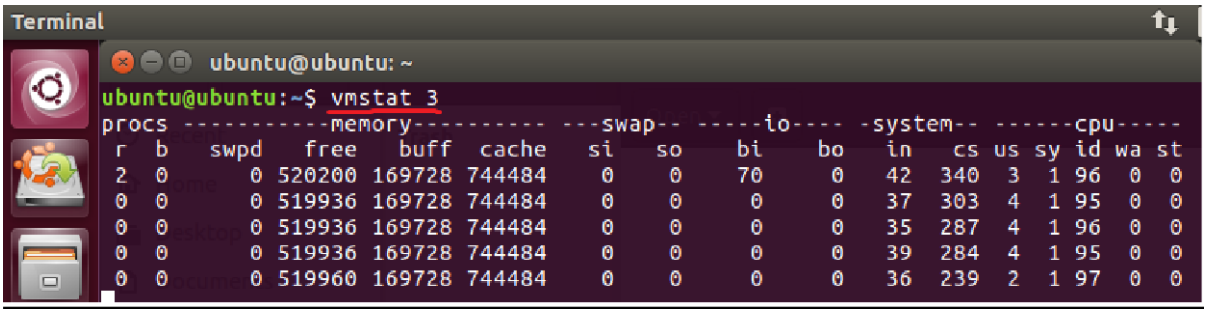


We can also use the renice command to change the priority of all of the processes belonging to a user or a group. In the following example, we change the nice value of all of the process belonging to user ubuntu.



Few more useful commands related to processes vmstat command

Outputs a snapshot of system resource usage including, memory, swap and disk I/O. To see a continuous display, follow the command with a time delay (in seconds) for updates. For example: vmstat 5.



xload command

Displays the system load over time.

tload command

This command is similar to xload but displays output in the terminal.

2) Lab Activities

Activity 1:

This activity is related to process monitoring in Linux

- ***Display all of the process in current shell***
- ***Display every active process in the system***
- ***Provide a full-format listing of process owned by you***
- ***Display a full-format listing of processes owned by user ubuntu***
- ***Display a process with id 1***
- ***Display the dynamic view of the current processes in the system and set the refresh interval 0.5 second***
- ***Display the dynamic view of the processes owned by user with id 999 (or name ubuntu)***
- ***Start the gedit program in background and then bring it foreground***
- ***Suspend the gedit program***
- ***Resume the gedit program***

Activity 2:

Perform the following tasks

- ***Start the gedit program with priority 10***

- ***Reset the priority of gedit to 15***